Introduction to Python

Why learn Python?

Python is increasingly the most popular choice of programming language for data analysts because it is designed to be simple, efficient, and easy to read and write.

There are many open source software and libraries that use Python and data analysis tools built on them.

We will use Python to learn programming and explore fundamental programming concepts of commands, variables, decisions, repetition, and events.

What is Python?

Python is a general, high-level programming language designed for code readability and simplicity.

Python is available for free as open source and has a large community supporting its development and associated tools.

Python was developed by Guido van Rossum and first released in 1991. Python 2.0 was released in 2000 (latest version 2.7), and a backwards-incompatible release Python 3 was in 2008.

- Our coding style will be Python 3 but most code will also work for Python 2.

Python Language Characteristics

Python supports:

- dynamic typing – types can change at run-time
- multi-paradigm – supports procedural, object-oriented, functional styles
- auto-memory management and garbage collection
- extendable – small core language that is easily extendable

Python core philosophies (by Tim Peters: https://www.python.org/dev/peps/pep-0020/)

- Beautiful is better than ugly
- Explicit is better than implicit
- Simple is better than complex
- Complex is better than complicated
- Readability counts

Some Quotes

"If you can't write it down in English, you can't code it."
-- Peter Halpern

"If you lie to the computer, it will get you."
-- Peter Farrar

Introduction to Programming

An ***algorithm*** is a precise sequence of steps to produce a result. A ***program*** is an encoding of an algorithm in a ***language*** to solve a particular problem.

There are numerous languages that programmers can use to specify instructions. Each language has its different features, benefits, and usefulness.

The goal is to understand fundamental programming concepts that apply to all languages.

Python: Basic Rules

To program in Python you must follow a set of rules for specifying your commands. This set of rules is called a ***syntax***.

- Just like any other language, there are rules that you must follow if you are to communicate correctly and precisely.

Important general rules of Python syntax:

- Python is ***case-sensitive***.
- Python is particular on whitespace and indentation.
- The end of command is the end of line. There is no terminator like a semi-colon.
- Use four spaces for indentation whenever in a block.

```
def spam():
    eggs = 12
    return eggs
print spam()
```

Comments

Comments are used by the programmer to document and explain the code. Comments are ignored by the computer. Two types:

- 1) One line comment: put "#" before the comment and any characters to the end of line are ignored by the computer.
- 2) Multiple line comment: put """ at the start of the comment and """ at the end of the comment. The computer ignores everything between the start and end comment indicators.

Example:

```
# Single line comment
print (1)  # Comment at end of line
""" This is a
multiple line
comment """
```

Python Programming

A Python program, like a book, is read left to right and top to bottom.

Each command is on its own line.

```
# Sample Python program
name = "Joe"
print("Hello")
print("Name: "+name)
```

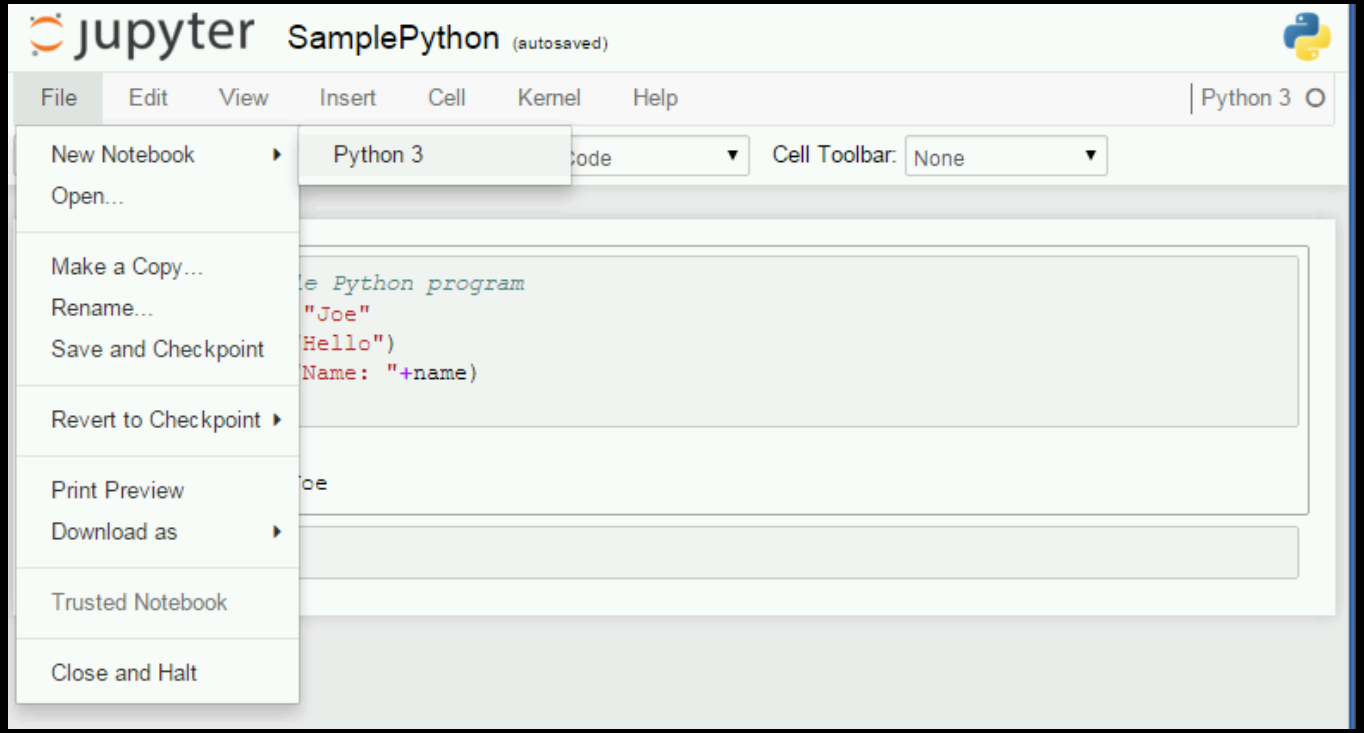
Flow of Execution
Start at first statement at top and proceed down executing each statement

A user types in a Python program in a text editor or development environment and then runs the program.

Python Editor - jupyter

jupyter is a graphical, browser-based editor for Python.

To create a new notebook, select `File`, `New Notebook`, `Python3`.



Python Editor – jupyter notebook

Button to run code (shortcut is Ctrl+Enter)

jupyter SamplePython (autosaved)

File Edit View Insert Cell Kernel Help Python 3

Code Cell Toolbar: None

In [2]:
```
# Sample Python program
name = "Joe"
print("Hello")
print("Name: "+name)
```

Python code

```
Hello
Name: Joe
```

Program output

In [ ]:

Python: Hello World!

Simplest program:

```
print("Hello World!")
```

The `print` function will print to the terminal (standard output) whatever data (number, string, variable) it is given.

Try it: Python Printing

Question 1: Write a Python program that prints "I am fantastic!".

Question 2: Write a Python program that prints these three lines:

```
I know that I can program in Python.
I am programming right now.
My awesome program has three lines!
```

Python Question

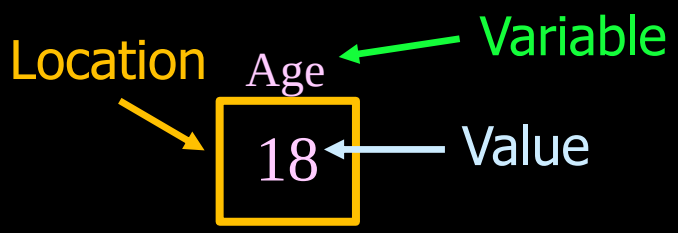
Question: How many of the following statements are **TRUE**?

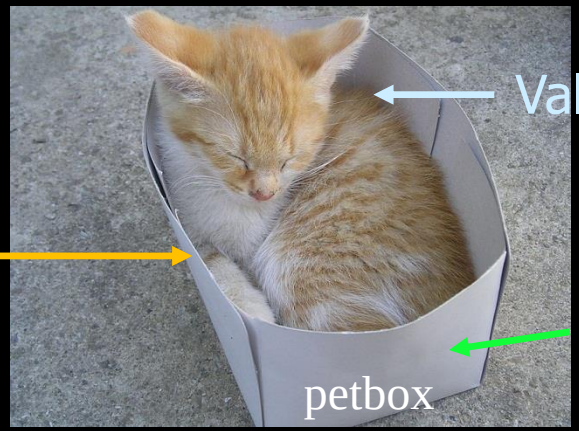
1) Python is case-sensitive.
2) A command in Python is terminated by a semi-colon.
3) Indentation does not matter in Python.
4) A single line comment starts with `"""`.
5) The `print` command prints to standard input.

A) 0 **B)** 1 **C)** 2 **D)** 3 **E)** 4

Variables

A ***variable*** is a name that refers to a location that stores a data value.





IMPORTANT: The *value* at a location can change using initialization or assignment.

Variable Assignment

Assignment using an **=** sets the value of a variable.

Example:

```
num = 10
message = "Hello world!"
```

num

10

message

Hello world!

Python Variables

To create a variable in Python, you must only provide a name.

- A variable type is dynamic. It can store numbers, strings, or Boolean at any time.

Example:

```
val = 5
val = "Hello"
```

Boolean values can be either `True` or `False`. Note case matters.

```
isAwesome = True
isAwesome = False
```

Variable Rules

Variables are a name that ***must begin with a letter*** and cannot contain spaces.

Variables are created when they are first used. There is no special syntax to declare (create) a variable.

Variable names **ARE** case-sensitive. Numbers are allowed (but not at the start). Only other symbol allowed is underscore ('_');

A programmer picks the names for variables, but try to make the names meaningful and explain their purpose.

Avoid naming variables as reserved words. A ***reserved word*** has special meaning in the language.

- e.g. `if`, `for`, `else`

Python Variables Question

Question: How many of the following variable names are valid?

1) `name`
2) `string2`
3) `2cool`
4) `under_score`
5) `spaces name`
6) `else`

A) 0 **B)** 1 **C)** 2 **D)** 3 **E)** 4

Python Math Expressions

Math ***expressions*** in Python:

Operation	Syntax	Example
Add	+	5 + 3
Subtract	-	10 – 2
Multiply	*	5 * 3
Divide	/	9 / 4
Modulus	%	9 % 4 (answer is 1)
Exponent	**	5 ** 2 (answer is 25)

Expressions - Operator Precedence

Each operator has its own priority similar to their priority in regular math expressions:

- 1) Any expression in parentheses is evaluated first starting with the inner most nesting of parentheses.
- 2) Exponents
- 3) Multiplication and division (*, /, %)
- 4) Addition and subtraction (+,-)

Python Expressions Question

Question: What is the value of this expression:

```
8 ** 2 + 12 / 4 * (3 - 1) % 5
```

A) 69 **B)** 65 **C)** 36 **D)** 16 **E)** 0

Try it: Python Variables and Expressions

Question 1: Write a program that prints the result of `35 + 5 * 10`.

Question 2: Write a program that uses at least 3 operators to end up with the value `99`.

Question 3: Write a program that has a variable called `name` with the value of your name and a variable called `age` storing your age. Print out your name and age using these variables.

Strings

Strings are sequences of characters that are surrounded by either single or double quotes.

- Use \ to escape ' E.g. There\'s
- Can use triple double quotes """ for a string that spans multiple lines.

Example:

```
name = "Joe Jones"
storeName = 'Joe\'s Store'
print("""String that is really long
with multiple lines
   and spaces is perfectly fine""")
```

Python String Indexing

Individual characters of a string can be accessed using square brackets (`[]`) with the first character at index 0.

Example:

```
str = "Hello"
print(str[1])        # e
print("ABCD"[0])     # A
print(str[-1])       # o
# Negative values start at end and go backward
```

Rules for Strings in Python

Must be surrounded by single or double quotes.

Can contain most characters except enter, backspace, tab, and backslash.

- These special characters must be escaped by using an initial "\".
- e.g. `\n` – new line, `\'` – single quote, `\\` - backslash, `\"` – double quote
- A string in raw mode (`r` before quote) will ignore backslash escape. May be useful if data contains escapes. Example: `st = r"slash\there\"`

Double quoted strings can contain single quoted strings and vice versa.

Any number of characters is allowed.

The minimum number of characters is zero `""`, which is called the *empty string*.

String *literals* (values) have the quotation marks removed when displayed.

Python Strings Question

Question: How many of the following are valid Python strings?

1) `""`
2) `''`
3) `"a"`
4) `" "`
5) `"""`
6) `"Joe\' Smith\""`

A) 1 **B)** 2 **C)** 3 **D)** 4 **E)** 5

Python String Functions

```
st  = "Hello"
st2 = "Goodbye"
```

Operation	Syntax	Example	Output
Length	`len()`	`len(st)`	`5`
Upper case	`upper()`	`st.upper()`	`HELLO`
Lower case	`lower()`	`st.lower()`	`hello`
Convert to a string	`str()`	`str(9)`	`"9"`
Concatenation	`+`	`st1 + st2`	`HelloGoodbye`
Substring	`[]`	`st[0:3]` `st[1:]`	`Hel` `ello`
String to int	`int()`	`int("99")`	`99`

String Operators: Concatenation

The ***concatenation operator*** is used to combine two strings into a single string. The notation is a plus sign '**+**'.

Example:

```
st1 = "Hello"
st2 = "World!"
st3 = st1 + st2 # HelloWorld!
print(st1+st1)
num = 5
print(st1+str(num)) # Hello5
# Must convert number to string before
# concatenation
```

String Concatenation Question

Question: What is the output of this code?

```
st1 = "Hello"
st2 = "World!"
num = 5
print(st1 + str(num) + " " + st2)
```

A) `Error`

B) `Hello5World!`

C) `Hello5 World!`

D) `Hello 5 World!`

Substring

The ***substring*** function will return a range of characters from a string.

Syntax:

```
st[start:end]   # start is included, end is not
                # first character is index 0
```

Examples:

```
st = "Fantastic"
print(st[1])          # a
print(st[0:6])        # Fantas
print(st[4:])         # astic
print(st[:5])         # Fanta
print(st[-6:-2])      # tast
```

Substring Question

Question: What is the output of this code?

```
st = "ABCDEFG"
print(st[1] + st[2:4] + st[3:] + st[:4])
```

A) ABCDCDEFGABCD

B) ABCDEFGABC

C) ACDDEFGABCD

D) BCDDEFGABCD

E) BCDECDEFGABC

Split

The ***split*** function will divide a string based on a separator.

Examples:

```
st = "Awesome coding! Very good!"
print(st.split())
# ['Awesome', 'coding!', 'Very', 'good!']
print(st.split("!"))
# ['Awesome coding', ' Very good', '']
st = 'data,csv,100,50,,25,"use split",99'
print(st.split(","))
# ['data', 'csv', '100', '50', '', '25',
#   '"use split"', '99']
```

Try it: Python String Variables and Functions

Question 1: Write a Python program that prints out your name and age stored in variables like this:

```
Name: Joe
Age: 25
```

Question 2: Write a Python program that prints out the first name and last name of `Steve Smith` like below. You must use substring.

- Bonus challenge: Use `find()` function so that it would work with any name.

```
First Name: Steve
Last Name: Smith
```

Print Formatting

The `print` method can accept parameters for formatting.

```
print("Hi", "Amy", ", your age is", 21)
print("Hi {}, your age is {}".format("Amy",21))
```

This is one of the most obvious changes between Python 2:

```
print "Hello"
```

and Python 3:

```
print("Hello")
```

Python Date and Time

Python supports date and time data types and functions.

First, import the `datetime` module:

```
from datetime import datetime
```

Functions:

```
now = datetime.now()
print(now)
current_year = now.year
current_month = now.month
current_day = now.day
print("{}-{}-{} {}:{}:{}".format(now.year, now.month,
now.day, now.hour, now.minute, now.second))
```

Python Clock

Python `time()` function returns the current time in seconds:

```
import time
startTime = time.time()
print("Start time:", startTime)
print("How long will this take?")
endTime = time.time()
print("End time:", endTime)
print("Time elapsed:", endTime-startTime)
```